NP-Completude

Motivação

Classe de problemas NP-completos (NPC):

- Contém muitos problemas.
- Nenhum algoritmo polinomial conhecido.
- Não há demonstração de que não existe algoritmo polinomial (em aberto desde 1971).
- Há forte evidência de que não existe algoritmo polinomial: se um problema NPC tiver algoritmo polinomial, então todos os problemas NP terão algoritmo polinomial.

Motivação

- Ao projetar algoritmos, buscamos obter tempo polinomial.
- Se não encontramos algoritmo polinomial depois de algum esforço, é sensato investigar se o problema é NPC.
- Mostrar que o problema é NPC significa que é improvável que exista algoritmo polinomial, portanto, não vale a pena continuar procurando algoritmo polinomial.
- Nos resta apenas buscar soluções aproximadas (heurísticas).

Problemas de decisão

- Os problemas da classe NPC são problemas de decisão, ou seja, problemas cujas respostas são "sim" ou "não".
- Todo problema de otimização pode ser transformado em um problema de decisão: basta perguntar se existe solução com valor melhor ou igual a um certo valor k.
- Exemplo:
  - Otimização: qual é o comprimento do menor caminho entre dois vértices s e t em um grafo?
  - Decisão: existe caminho entre s e t com comprimento menor ou igual a k?

Problemas de decisão

- Se a versão de otimização tem algoritmo polinomial, então a versão de decisão também tem. Basta testar se o valor da solução ótima é menor ou igual a k.
- Portanto, se a versão de decisão não tem algoritmo polinomial, então a versão de otimização também não tem.
- Assim, mesmo que seu problema de interesse seja de otimização, basta mostrar que a versão de decisão deste problema é NPC.

Classe P

- Problemas de decisão que podem ser resolvidos em tempo polinomial.
- Exemplo: "existe caminho entre s e t em um grafo?" pode ser resolvido em tempo polinomial usando busca em largura ou busca em profundidade.

Classe NP

- Problemas de decisão cujas soluções podem ser verificadas em tempo polinomial.
- Exemplo: "existe um ciclo que passa exatamente uma vez por cada vértice (hamiltoniano)?" é um problema NP, pois se alguém fornecer tal ciclo, podemos verificar se ele é válido em tempo polinomial.
- Todo problema na classe P também está na classe NP, pois podemos determinar se a instância é "sim" ou "não" em tempo polinomial, sem o auxílio de uma solução fornecida.

Classe NP

- NP significa “polinomial em máquina não determinística”.
  - Máquinas não determinísticas são capazes de explorar simultaneamente vários possíveis caminhos até a solução (não existem).
  - Teorema: verificado em tempo polinomial em máquina determinística $\iff$ resolvido em tempo polinomial em máquina não determinística.

Redução polinomial

- Um problema A é redutível a um problema B se uma instância de A pode ser transformada em uma instância de B em tempo polinomial, de modo que a resposta da instância de A seja "sim" se e somente se a resposta da instância de B for "sim".
- Assuma que A é redutível a B.
  - Se B tem algoritmo polinomial, então podemos resolver A em tempo polinomial.
  - Portanto, se A não tem algoritmo polinomial, então B também não tem algoritmo polinomial.

Classe NPC (NP-completo)

- Problemas de decisão em NP tal que todo problema em NP é redutível a eles.
- Portanto, são os problemas mais difíceis da classe NP, pois algoritmo polinomial para qualquer deles implicaria algoritmo polinomial para todos os problemas em NP.

P = NP?

- Em aberto desde 1971.
- Para provar P=NP, bastaria encontrar um algoritmo polinomial p/ qualquer prob. NPC.
  - Nunca encontrado nem provado que não existe.
- Ou seja, não sabemos ainda se verificar soluções é mais fácil do que encontrar soluções!
- Exemplos de impactos: criptografia, otimização, bioinformática, etc.

Como mostrar que um problema B é NPC?

1. Mostre que o problema B está em NP (verificação em tempo polinomial).
2. Escolha um problema A já conhecido como NPC.
3. Mostre que A é redutível a B.

- A escolha de A impacta a dificuldade/viabilidade do passo 3.
- Conhecer muitos problemas NPC ajuda a escolher o A mais adequado (Garey e Johnson).
- Precisamos de um primeiro problema NPC!

Primeiro problema NPC

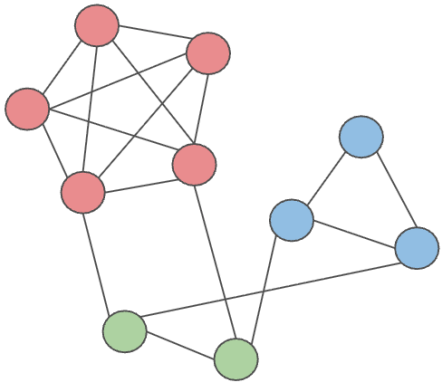
- Primeiro problema provado NPC foi o SAT (satisfatibilidade de fórmulas booleanas).
- Provado por Stephen Cook em 1971.
- SAT: dado uma fórmula booleana, existe uma atribuição de valores verdadeiro/falso às variáveis que torna a fórmula verdadeira?
- Ex.: $(x_1 \vee \neg x_2) \wedge (x_2 \vee x_3)$ é satisfatível, pois $x_1 = V, x_2 = F, x_3 = V$ torna verdadeira.

Ex.: Redução de SAT para 3-SAT

- 3-SAT: dado uma fórmula booleana em forma normal conjuntiva (CNF), existe uma atribuição de valores verdadeiro/falso às variáveis que torna a fórmula verdadeira, tal que cada cláusula tem exatamente 3 literais?
- Ex.: $(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4)$ é satisfatível.
- Redução: existe algoritmo polinomial que transforma qualquer fórmula booleana em fórmulas CNF equivalentes com cláusulas de tamanho 3.
  1. Transforme em CNF aplicando distributiva. Ex.: $A \vee (B \wedge C) = (A \vee B) \wedge (A \vee C)$.
  2. Se uma cláusula tem menos de 3 literais, adicione variáveis auxiliares para completar até 3. Ex.: $(x_1 \vee x_2)$ transformada em $(x_1 \vee x_2 \vee y) \wedge (x_1 \vee x_2 \vee \neg y)$, onde y é uma nova variável.
  3. Se tem mais de 3 literais, divida em várias cláusulas de 3 literais usando variáveis auxiliares. Ex.: $(x_1 \vee x_2 \vee x_3 \vee x_4 \vee x_5)$ transformada em $(x_1 \vee x_2 \vee y_1) \wedge (\neg y_1 \vee x_3 \vee y_2) \wedge (\neg y_2 \vee x_3 \vee x_4)$, onde y_1 e y_2 são novas variáveis.

Ex.: Clique em grafos

- Clique: conjunto de vértices tal que todos os pares de vértices estão conectados p/ aresta.
- Problema: dado um grafo G e um inteiro k, existe uma clique de tamanho k em G?

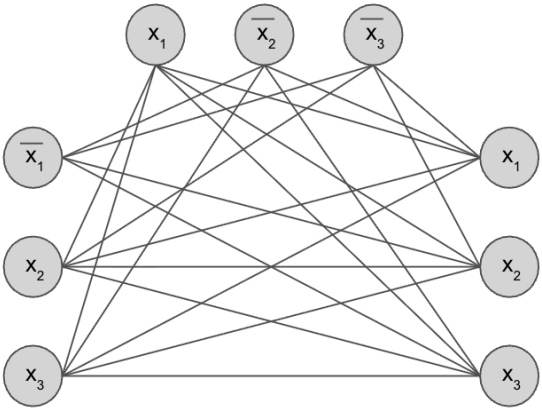


Ex.: Clique em grafos

- Pertence a NP: dada solução, basta verificar #vértices e se todos pares estão conectados.
- Redução: reduzir 3-SAT para Clique.
  1. Um nó para cada literal.
  2. Uma aresta entre dois nós se eles não forem da mesma cláusula e não forem complementares.
  3. $k = \#\text{cláusulas}$.
     - Aresta indica que par de literais de cláusulas distintas pode ser satisfeito simultaneamente.
     - Fórmula satisfatível $\iff$ existe clique de tamanho k.

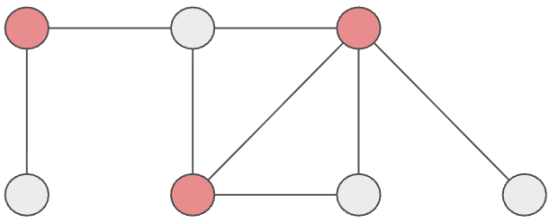
Ex.: Clique em grafos

- Ex.: $(x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$



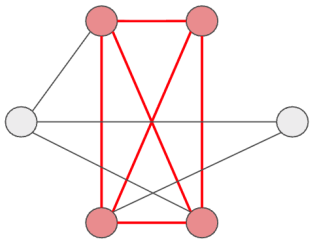
Ex.: Cobertura por vértices

- Cobertura por vértices: conjunto de vértices tal que cada aresta do grafo tem pelo menos um dos seus extremos no conjunto.
- Problema: dado um grafo G e um inteiro k, existe uma cobertura por vértices de tamanho no máximo k em G?

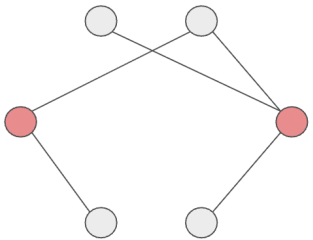


Ex.: Cobertura por vértices

- Pertence a NP: verificar #vértices e se todas as arestas incidem no conjunto.
- Redução: reduzir Clique (grafo G) para Cobertura por vértices ($\overline{G}$).
  1. Grafo complementar $\overline{G}$: mesmo nós, e apenas as arestas que não ocorrem em G.
     - Clique de tam. pelo menos k em $G \iff$ cobertura de tam. no máx. $|V| - k$ em $\overline{G}$



Clique em G



Cobertura no complemento de G